

When Exact Search Wins—and When It Does Not: Evidence from Predicate-Aware Vector Retrieval

Akshat Singh Kushwaha

akshatsingh14372@outlook.com

<https://www.linkedin.com/in/akshat-singh-kushwaha>

September 2026

Abstract

Metadata predicates can turn a million-row nearest-neighbor query into a scan of only thousands of eligible vectors, changing the choice between exact CPU, exact GPU, and approximate search. We audit Qenlo, an embedded vector-search prototype, and synthesize three retained cohorts plus a user-supplied Android device-lab cohort rather than claiming a universal winner. On a real $100k \times 384$ Windows workload, Qenlo’s shipped WGPU exact path reduced P95 from 16.657 ms on CPU to 3.240 ms when all rows were eligible, but CPU won at 1% eligibility. A post-hoc six-point sweep placed the compact-row CPU/WGPU winner reversal between 2,000 and 3,000 eligible rows. On Android, a 512-sample $100k \times 384$ soak likewise favored WGPU exact over CPU at P95 (17.678 versus 27.019 ms), while the 10k quick cell favored CPU. On the Windows dense cell WGPU was nominally faster than tuned USearch (3.625 ms at recall 0.99224), although the bootstrap interval crossed parity. On a real $1M \times 768$ TopK workload with 10,026 eligible rows, FAISS GPU Flat, NumPy exact, and a transparent Qenlo CUDA prototype achieved P95 latencies of 0.132, 0.197, and 0.171 ms, respectively, all at recall@10=1.0. A fresh-seed synthetic A6000 experiment found that the buffered CUDA prototype lost to FAISS at 1k and 10k eligible vectors but won at 100k (0.567 versus 0.789 ms) and 1M (4.668 versus 6.347 ms), with complete top-10 agreement. Exact GPU search therefore has a measurable operating region, but missing HNSW curves, a full batch matrix, and native-WGPU A6000 cells leave the broad hypothesis only partially supported.

1 Introduction

Approximate nearest-neighbor (ANN) indexes are commonly treated as the default response to large vector collections. A metadata predicate invalidates the simplifying premise that every stored vector is a candidate. If a predicate produces an eligible set S_P , exact top- k cosine retrieval costs approximately $O(|S_P|D)$ rather than $O(ND)$. At sufficiently small $|S_P|$, maintaining or traversing a graph can be unnecessary; at very small $|S_P|$, GPU launch and synchronization overhead can make a CPU scan preferable.

This paper asks: *when metadata predicates substantially reduce the eligible set, when is exact vector search preferable to ANN?* We use Qenlo as an experimental vehicle because it places canonical rows, exact CPU search, filtered HNSW, portable GPU exact search, and observable routing behind one in-process interface. Our starting hypothesis, H1, is that predicate-aware exact GPU search can beat CPU exact and become competitive with tuned high-recall HNSW while retaining exact recall. Secondary hypotheses concern eligible-count dominance (H2), a CPU/GPU crossover (H3), batching (H4), the high-recall ANN tradeoff (H5), and adaptive routing (H6).

The contribution is chiefly an evidence audit. First, we map implemented behavior to source and tests. Second, we distinguish three non-poolable cohorts and preserve failures. Third, we expose two reversals: CPU versus WGPU as selectivity changes, and FAISS versus a buffered CUDA formulation as eligible count grows. Fourth, we identify what the current evidence cannot answer. This is not a claim that Qenlo is production proven or that exact search universally dominates ANN.

2 Background and motivation

For normalized vectors, cosine ranking is inner-product ranking. Exact search evaluates every eligible vector and returns the globally ordered top k . IVF reduces work by probing partitions, and product or scalar quantization reduces representation cost, potentially sacrificing recall. HNSW constructs a navigable proximity graph and exposes

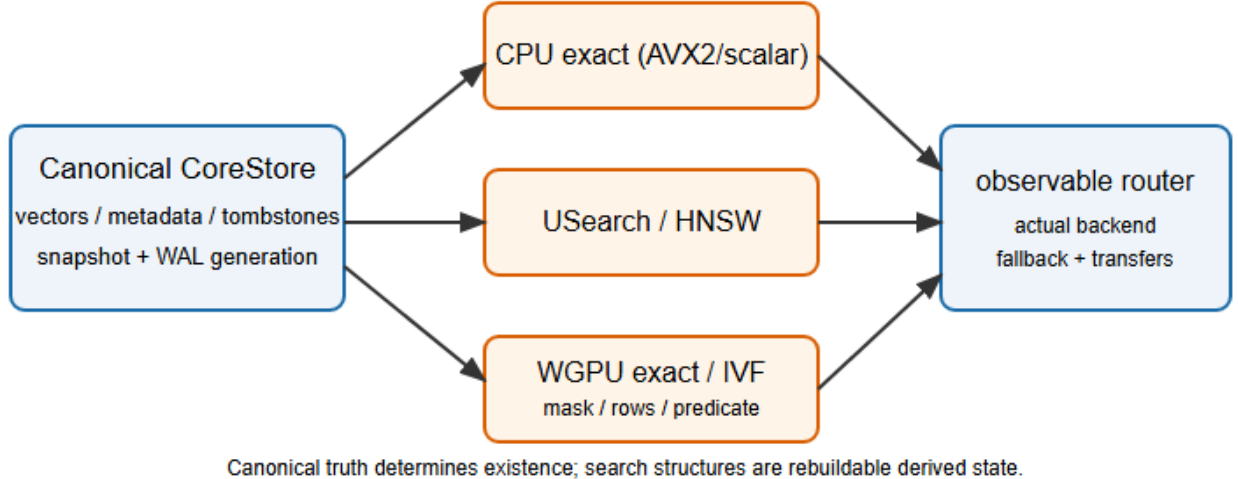


Figure 1: Canonical rows determine logical existence. Search structures are disposable derived state.

a search-expansion parameter governing a recall/latency tradeoff [5]. FAISS provides both exact and approximate CPU/GPU implementations [4, 2].

Filtering complicates ANN because graph edges ignore arbitrary query predicates. Post-filtering can return too few results; pre-filtering may disconnect useful graph paths. Systems such as Filtered-DiskANN and ACORN modify graph construction or traversal to address this interaction [3, 6]. Exact prefiltered search avoids an ANN recall parameter: its cost is driven by eligible work, but candidate materialization and device transfer remain important.

We distinguish four notions often conflated in reports: total rows N , eligible rows E , scored rows, and bytes transferred. We also distinguish GPU-event kernel time from host-observed completed-call time. Only the latter includes dispatch, synchronization, and result readback.

3 Qenlo

Qenlo’s **CoreStore** owns normalized FP32 vectors, metadata, public IDs, permanent slots, tombstones, and the committed generation. User and timestamp indexes are reconstructed from canonical rows. A collection wraps this state with a read/write lock and owns derived CPU, HNSW, and GPU state. Figure 1 shows the ownership boundary.

CPU exact search first identifies eligible rows, uses AVX2/FMA when available (scalar otherwise), accumulates in FP64, and retains at most k hits. USearch performs filtered HNSW with live canonical IDs. The WGPU path supports a CPU mask, a compact eligible-row list, or direct GPU predicate evaluation. It scans bounded chunks and returns per-chunk top- k lists for a host merge. If an element is absent from its chunk top k , at least k elements in that chunk precede it, so it cannot enter the global top k ; the merge is exact under a common distance/ID ordering.

The WGPU module also implements IVF-Flat and IVF-SQ8 candidate generation followed by exact FP32 reranking. These remain approximate when unprobed lists omit a true neighbor. Automatic routing and required-GPU modes are distinct: required mode reports failure, whereas automatic mode may fall back and records that fact.

Durability supports the search story but is not the performance contribution. Mutations produce immutable checksummed WAL generations and an atomic checksummed **HEAD**; flush/close compact to snapshots. Tests cover

tombstones, ordered WAL replay, corruption, uncertain publication, and recovery. The system claims process-crash recovery under documented filesystem behavior, not universal sudden-power-loss durability.

4 Methodology

4.1 Cohorts and non-pooling rule

Table 1 lists the evidence. Cohorts differ in hardware, data, implementation, and timing boundary, so we compare within a cohort and use cross-cohort observations only as context.

Table 1: Evidence cohorts. “Native” means repository Rust code.

Cohort	Data	Hardware	Systems	Samples/cell
Windows retained	real AG News, 100k×384	RTX 4050	native CPU/WGPU	5 runs
Android device lab	generated, 10k/100k×384	MT6897/Mali-G615	native CPU/WGPU/IVF	16–512
TopK retained	real MS MARCO, 1M×768	RTX A6000	NumPy/FAISS/CUDA	5,000
New held-out	synthetic, 1M×768	RTX A6000	FAISS/CUDA	500

The public TopK corpus uses ModernBERT embeddings and an integer filter [8]. Retained checksums are `f1c9f0...275d3` for documents and `d9a34c...00870` for queries. Its independently computed FP64 oracle is outside timed search. The new matrix uses normalized random FP32 vectors, 100 queries, 50 warmups, five repetitions, and seeds 20260902 (development) and 20260903 (held out). Queries are shuffled per repetition. The measured boundary starts before the system call and ends after ten IDs return to the host; corpus/filter preparation is excluded. Qenlo and FAISS search the identical prefiltered matrix.

The A6000 was a 49,140 MiB RTX A6000, driver 570.195.03, CUDA 12.8, PyTorch 2.9.1, and FAISS 1.14.1. The Runpod campaign used one GPU at a time, cost \$0.9927, and left zero pods. Native WGPU attempts failed because the container had no usable NVIDIA Vulkan ICD; those attempts produced no latency samples.

The author-supplied Android schema-v1 records identify Android 16/API 36, a Mediatek MT6897 CPU, Mali-G615 MC6 GPU, Vulkan, and app version 0.1.0. Quick, full, and soak suites contain 16, 64, and 512 samples per cell. All 21 cells report $\text{recall@10}=1.0$, no fallback, and no failure. We transcribed the records without pooling suites. Power source is absent and thermal state is the uninterpreted string “0”, so the mobile evidence has no controlled power or thermal claim.

4.2 Correctness and statistics

The repository oracle exhaustively scores eligible vectors in independent FP64 code outside timing. It rejects duplicate, deleted, unknown, or predicate-ineligible results. The TopK run recorded all 5,000 exact samples at $\text{recall@10}=1.0$. The new synthetic matrix uses unordered top-10 agreement between two exact implementations; all 400 query/cell checks agree. This is weaker than an independent oracle and is labeled accordingly.

We report P50/P95/P99 over raw completed calls. The retained Rust headline uses the median of five per-run P95 values and a seeded whole-run bootstrap interval, preserving run dependence. The synthetic table reports pooled descriptive percentiles; we do not call its differences statistically significant.

5 Results

5.1 CPU/GPU reversal under filtering

On the retained real 100k×384 cohort, all-row Qenlo WGPU predicate search achieved 3.240 ms P95 versus 16.657 ms for exact CPU, a 5.14× ratio with bootstrap interval [3.55,6.77]. At 1% eligibility, CPU required 0.230 ms and WGPU predicate evaluation 2.883 ms. A subsequent post-hoc sweep compared CPU with compact-row WGPU at $E \in \{2k, 3k, 4k, 6k, 8k, 10k\}$. Each cell used 5,000 held-out queries, 200 warmups, five repetitions, and $\text{recall@10}=1.0$. Median-run P95 was 0.520 versus 0.634 ms at $E = 2k$, then 0.956 versus 0.814 ms at $E = 3k$; WGPU remained faster through $E = 10k$. Thus the observed native winner reversal is bracketed to $(2k, 3k)$ for this implementation and machine. Because the sweep followed inspection of endpoint cells, it is localization evidence rather than a preregistered confirmation.

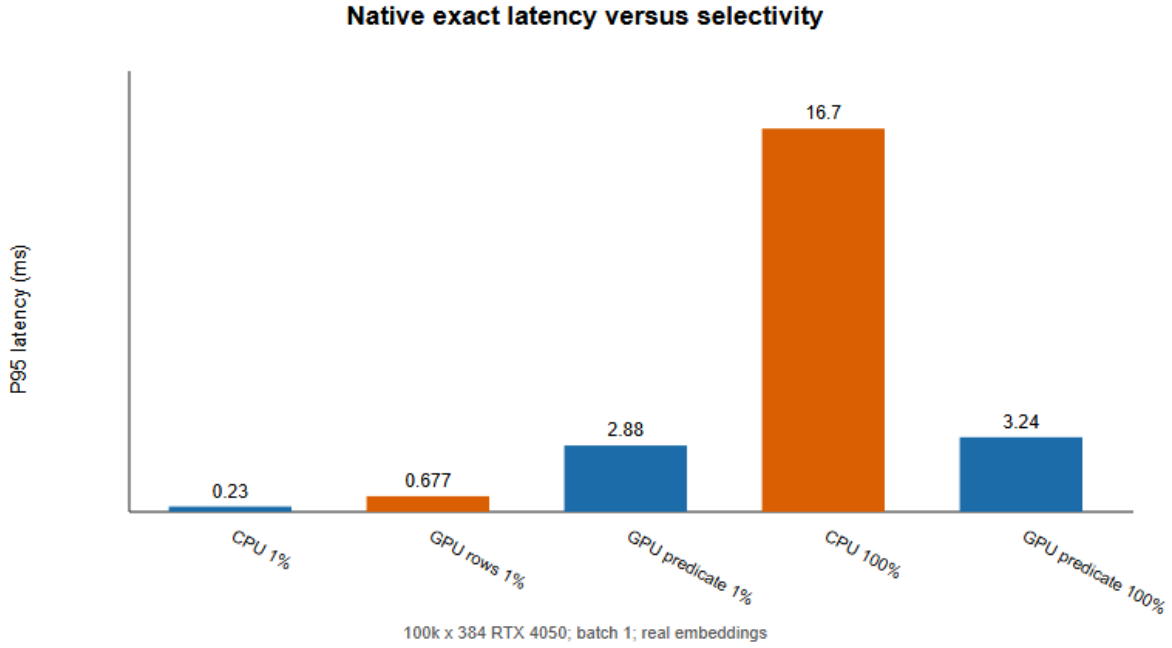


Figure 2: Native exact-search P95 on the controlled RTX 4050 cohort. CPU wins the selective cell; WGPU wins the dense cell.

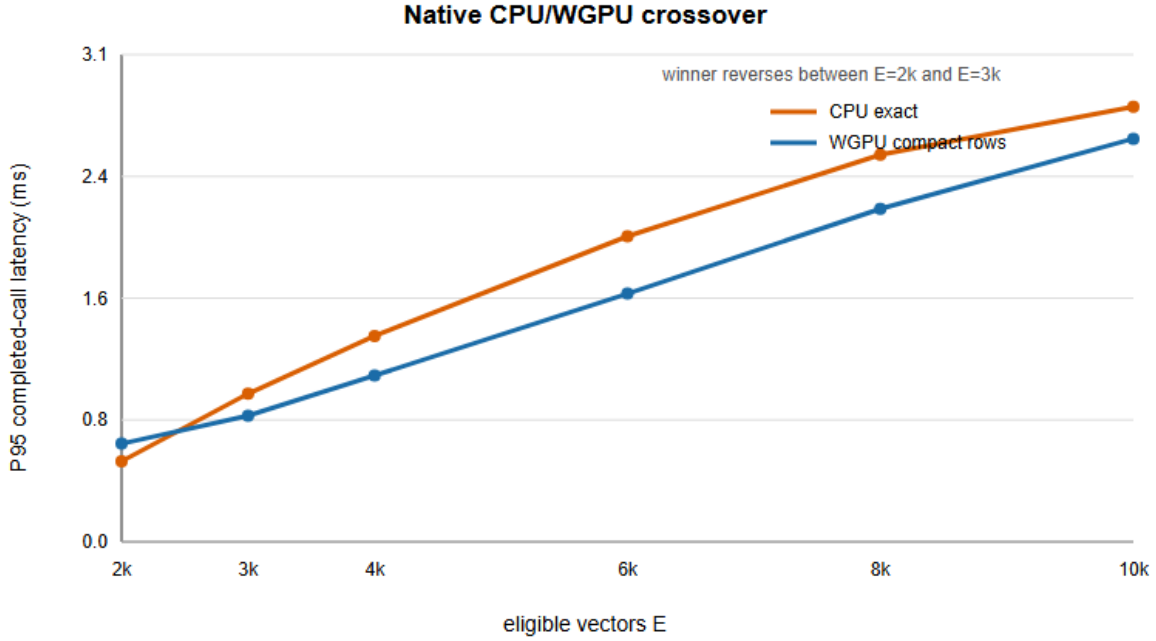


Figure 3: Post-hoc native crossover sweep on the RTX 4050, batch 1. CPU and compact-row WGPU use identical eligible sets; every cell has recall@10=1.0.

5.2 Android device-lab evidence

The mobile runs reproduce the qualitative reversal on a different architecture. In the 10k-row quick suite, CPU exact beat WGPU exact at P95 (5.767 versus 12.487 ms). In the 100k-row full suite WGPU won (16.975 versus 29.436 ms), and the 512-sample soak retained that result (17.678 versus 27.019 ms), a descriptive 1.53× CPU/WGPU ratio. This is not a controlled crossover bracket because suite length and row count change together. In the soak, reported

P95 values were 11.131 ms for native WGPU batch 8, 21.453 ms for IVF-Flat, and 37.090 ms for IVF-SQ8; all had recall 1.0. The batch-8 statistic is reported as emitted and is not normalized to per-query latency.

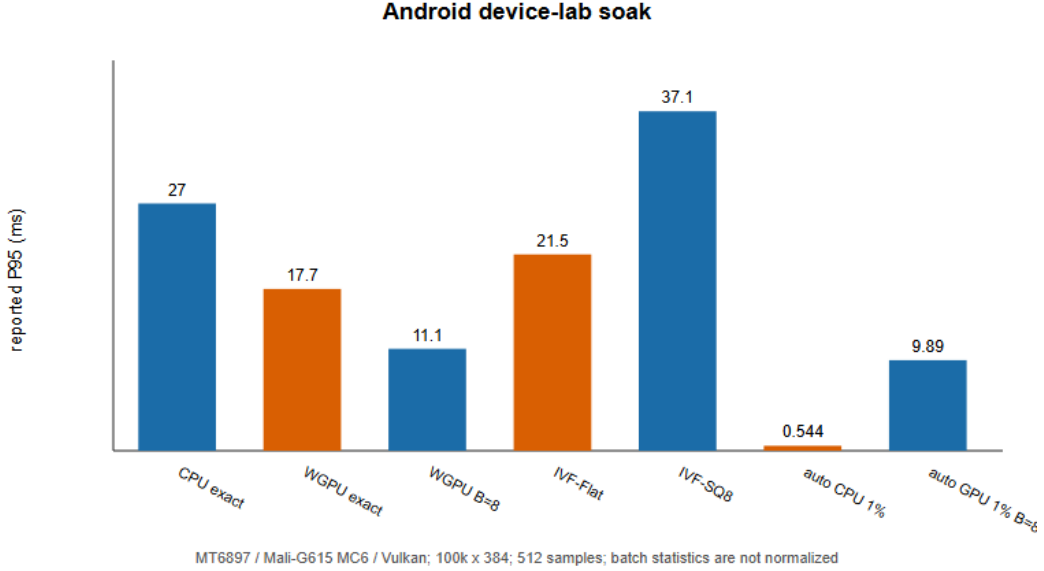


Figure 4: User-supplied Android soak results, 100k×384 and 512 samples per cell. Batch-1, batch-8, dense, and selective cells are shown together for inventory, not treated as direct pairwise speedups.

5.3 Real 1M×768 strict-filter cell

At $E = 10,026$, Table 2 shows that FAISS GPU Flat is fastest. All qualified systems are exact against the independent oracle. The CUDA prototype is $1.29\times$ slower than FAISS at P95. The previously noted 0.1196 ms is a different, synthetic kernel-only measurement and is not used here.

Table 2: Real TopK 1M×768, 1% predicate, batch 1, $k = 10$.

System	P50 ms	P95 ms	Recall@10
NumPy exact CPU	0.122	0.197	1.000
FAISS GPU Flat	0.126	0.132	1.000
Qenlo CUDA prototype	–	0.171	1.000

5.4 FAISS crossover on held-out synthetic data

Table 3 and the retained SVG plot show a second boundary. FAISS wins at 1k and 10k; the buffered matrix-vector/top-k formulation wins at 100k and 1M. The observed crossover is bounded only to $(10k, 100k)$ because intermediate points were not measured.

Table 3: Held-out A6000 P95 completed-call latency (ms), 1M×768 synthetic corpus.

Eligible E	FAISS Flat	Qenlo CUDA prototype	Faster
1,000	0.086	0.098	FAISS
10,000	0.137	0.156	FAISS
100,000	0.789	0.567	prototype ($1.39\times$)
1,000,000	6.347	4.668	prototype ($1.36\times$)

Latency rises monotonically and plausibly with E , addressing the most basic cached-answer and missing-work sanity checks. The reversal is an implementation result, not evidence that PyTorch generally outperforms FAISS or that native Qenlo does so.

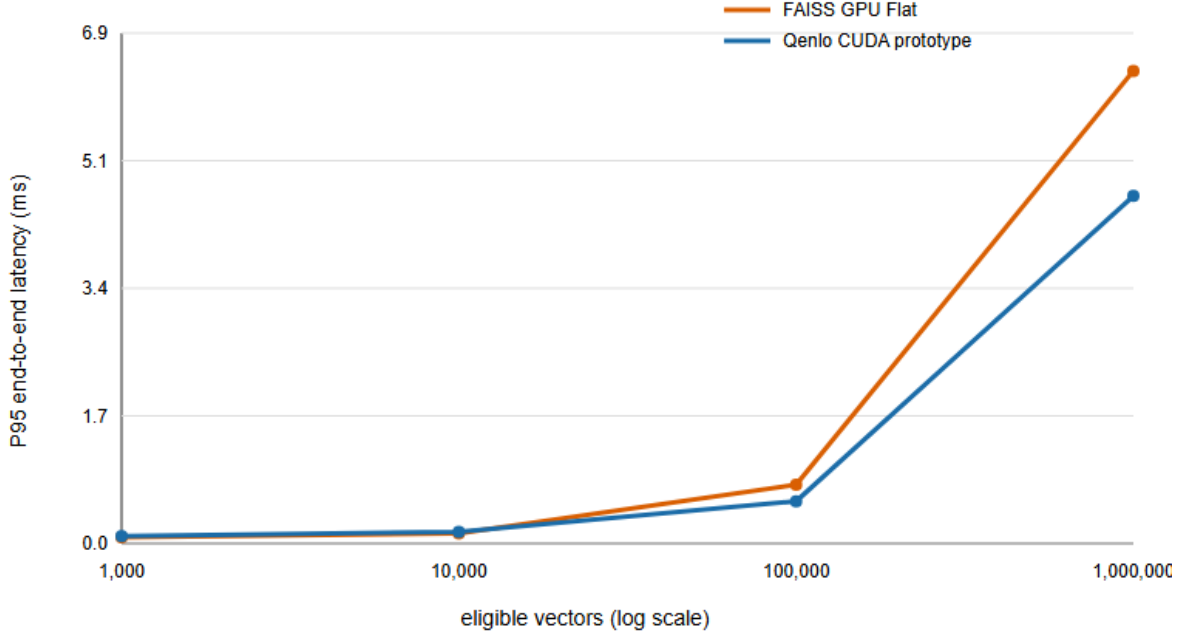


Figure 5: Held-out synthetic A6000 exact-search crossover. Both implementations return results to the host.

5.5 Exact versus ANN

The controlled Windows 100k×384 all-row cohort used identical data, query ranges, in-process boundary, and hardware. USearch expansion 128 was selected using 1,000 tuning queries, where recall was 0.9932. On 5,000 held-out queries repeated five times, USearch achieved P95 3.625 ms at recall 0.99224; exact WGPU achieved 3.240 ms at recall 0.99998. The nominal exact-over-ANN advantage is $1.12\times$, but its whole-run bootstrap interval $[0.77, 1.54]$ crosses parity. We therefore establish competitiveness, not a reliable latency winner. This is the strongest controlled support for H1/H5. A separate Linux cohort (USearch 5.093 ms at recall 0.9914, FAISS HNSW 1.494 ms at 0.9994) is context only because its successful Qenlo GPU cell is absent.

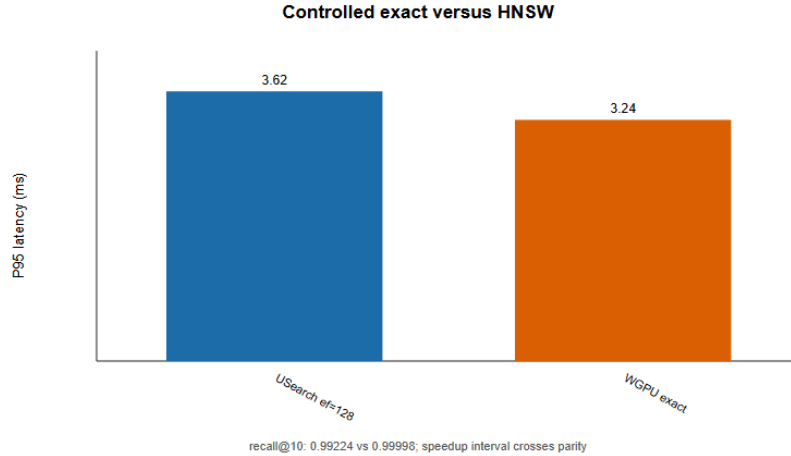


Figure 6: One controlled high-recall ANN operating point. The uncertainty interval for the latency ratio crosses parity.

6 Ablation and routing implications

The available selectivity ablation explains fixed costs. Native predicate WGPU changes little between 100% and 1% (3.240 versus 2.883 ms) while CPU scan cost collapses (16.657 versus 0.230 ms). At 1%, compact eligible-row WGPU reduces P95 to 0.677 ms, a $4.26\times$ improvement over direct GPU predicate evaluation, but CPU remains $2.94\times$ faster. At 100%, predicate, rows, and mask cost 3.240, 4.243, and 4.459 ms. The CUDA prefiltered formulation scales with E but excludes candidate materialization, reinforcing that compaction cost and reuse determine the boundary.

For batch 1 on the Windows cohort, the measured compact-row boundary suggests CPU through $E = 2k$ and GPU from $E = 3k$ in the sampled region; unsampled values and other hardware still require conservative handling. A retained Intel Arc soak run shows the router choosing CPU for $E = 1,000$, batch 1 (0.273 ms P95), and GPU for the same E across batch 8 (0.555 ms per reported query statistic); all samples had recall 1.0 and no fallback. Dense GPU P95 fell from 4.444 ms at batch 1 to 1.245 ms in the reported batch-8 statistic. These 512-sample device-lab cells support H4 qualitatively. They do not provide both fixed-policy counterfactuals for each cell, so routing accuracy and regret remain uncomputed rather than inferred.

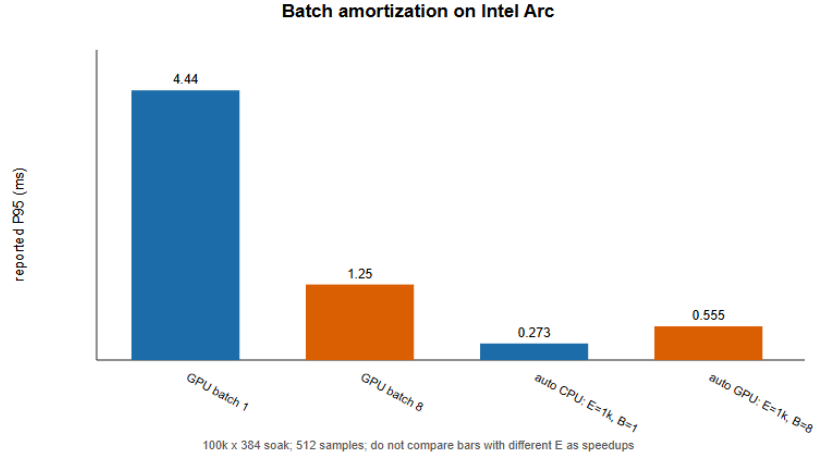


Figure 7: Batch evidence from the Intel Arc soak. Bars with different eligible counts are context, not direct speedups.

7 Correctness and system semantics

Canonical existence is independent of derived indexes. A successful mutation advances a generation; rebuildable indexes may lag only through an explicit not-prepared state, not by changing logical membership. Searches hold a shared lock across a committed generation; mutation and rebuild hold the write lock. A stale HNSW graph is filtered through canonical live IDs. Exact predicate tests cover conjunction, ranges, deletion, and independently-oracled search. These properties justify using a common oracle, but tests are not a proof of every crash or concurrency schedule.

8 Discussion

The phrase “exact search wins” needs two qualifiers. First, the relevant unit is eligible scoring work, approximately EDB , only when the implementation actually compacts or avoids ineligible work. Second, fixed costs and library kernels matter: at small E , FAISS’s exact GPU path beats the prototype; in native Rust, CPU beats WGPU at 1% eligibility. At large E , a dense GEMV plus top-k can outperform the tested FAISS Flat call, but it remains linear and loses ANN’s sublinear-work opportunity.

The strongest supported system lesson is therefore adaptive rather than competitive. Exact CPU is compelling for tiny eligible sets. Exact GPU is compelling for dense eligible work where transfer and dispatch amortize. High-recall ANN can be compelling when E remains large, but must be tuned on separate queries and reported with recall. Total corpus size alone is insufficient; it affects resident memory and predicate evaluation even when E controls scoring.

9 Limitations and threats to validity

Only an RTX 4050, Mali-G615 MC6, and RTX A6000 contribute successful GPU evidence, with different OSes, drivers, APIs, CPUs, and datasets. The Android records are author-supplied device-lab output rather than independently rerun campaign data; power source and interpretable thermal state are absent, and no raw per-sample distribution is available beyond reported percentiles. The six-point Windows crossover sweep is post-hoc and reports descriptive medians of per-run P95, without an uncertainty interval for the crossing location. The principal A6000 crossover is synthetic and uses a CUDA/PyTorch prototype rather than Qenlo’s shipped WGPU backend. The real TopK dataset is one embedding model and one independent integer predicate. The full requested dimensions, batch sizes, matched absolute eligible counts, and positive/negative/skewed correlations were not measured. No concurrent-client, energy, stable power, or comprehensive memory experiment was run. Build and rebuild time is outside query latency. The new experiment compares exact implementations, not ANN. The controlled Windows HNSW comparison has only one expansion point and five repetitions; its wide interval includes parity. One CUDA seed is held out, but hardware variability is not sampled across machines. Driver and WGPU overhead may dominate native results. Results concern cosine on normalized FP32 vectors and need not transfer to other metrics or precision. In-process calls exclude RPC and service scheduling. Durability tests do not include a physical power-cut campaign.

10 Related work

HNSW established a strong graph-based ANN design [5]; DiskANN and SPANN study billion-scale memory/storage hierarchies [7, 1]. FAISS systematizes exact, IVF, quantized, and GPU search [4, 2]. Filtered-DiskANN and ACORN directly address predicate-aware graph traversal [3, 6]. Our narrower question is the execution boundary at which a filter makes exhaustive scoring competitive, with canonical storage and derived indexes held conceptually separate.

11 Conclusion

Approximation is not universally necessary, and GPUs are not universally faster. Real native Windows evidence localizes a compact-row CPU/WGPU winner reversal to between 2,000 and 3,000 eligible vectors for 384 dimensions and batch 1 on the tested RTX 4050. User-supplied Android evidence independently shows CPU winning a 10k quick cell and WGPU winning both 100k full and soak cells; the soak advantage is $1.53\times$ at P95. In a controlled Windows dense cell, exact WGPU was nominally $1.12\times$ faster than tuned USearch at 0.992 recall, although uncertainty includes parity. On a held-out synthetic A6000 workload, a buffered exact CUDA prototype crosses FAISS GPU Flat between 10k and 100k eligible 768-dimensional vectors and is $1.36\text{--}1.39\times$ faster at the two larger points. Yet FAISS wins the real selective TopK cell. H1 is partially supported; H3 is supported across desktop and mobile observations; H2 and H4 are suggestive; H5 is supported in one cell; H6 remains open. The defensible outcome is a bounded map, not a universal Qenlo victory.

References

- [1] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. SPANN: Highly-efficient billion-scale approximate nearest neighborhood search. In *Advances in Neural Information Processing Systems*, volume 34, pages 5199–5212, 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/hash/299dc35e747eb77177d9cea10a802da2-Abstract.html.
- [2] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The Faiss library. *arXiv preprint arXiv:2401.08281*, 2024. URL <https://arxiv.org/abs/2401.08281>.
- [3] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, and Yin Zhang. Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters. In *Proceedings of the ACM Web Conference 2023*, pages 3406–3416, 2023. doi: 10.1145/3543507.3583552.
- [4] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021. doi: 10.1109/TBDATA.2019.2921572.

- [5] Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020. doi: 10.1109/TPAMI.2018.2889473.
- [6] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. ACORN: Performant and predicate-agnostic search over vector embeddings and structured data. *Proceedings of the ACM on Management of Data*, 2(3), 2024. doi: 10.1145/3654923.
- [7] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. DiskANN: Fast accurate billion-point nearest neighbor search on a single node. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL <https://proceedings.neurips.cc/paper/2019/hash/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Abstract.html>.
- [8] TopK. TopK Bench: Vector database benchmarks. <https://github.com/topk-io/bench>, 2026. Commit 398ce7d72dea7ad765ada54c5daa014c498efb52; accessed 2026-09-02.

A Artifact inventory

Raw A6000 and native-crossover samples are under `research/data/raw`; processed tables are under `research/data/processed`. The repository’s benchmark-results directory contains the independently-oracled TopK cohort and retained multi-backend archive. Exact leaf names and commands are listed in `paper/REPRODUCE.md`.

B Native crossover values

E	CPU P95	WGPU-row P95	CPU/WGPU
2,000	.520	.634	.82
3,000	.956	.814	1.17
4,000	1.331	1.074	1.24
6,000	1.974	1.603	1.23
8,000	2.500	2.151	1.16
10,000	2.810	2.604	1.08

C Complete held-out CUDA percentiles

System	E	P50	P95	P99 (ms)
FAISS	1,000	.067	.086	.095
Prototype	1,000	.078	.098	.115
FAISS	10,000	.127	.137	.151
Prototype	10,000	.144	.156	.174
FAISS	100,000	.775	.789	.800
Prototype	100,000	.556	.567	.576
FAISS	1,000,000	6.274	6.347	6.436
Prototype	1,000,000	4.651	4.668	4.736

D Failure record

Three pod/image attempts failed to provide usable WGPU: a PyTorch image exposed CUDA but Vulkan selected software or could not load the NVIDIA ICD; NVIDIA Vulkan images either lacked SSH or used an obsolete unsigned package repository. A 5.56 GiB public dataset transfer was aborted after throughput made it economically inappropriate. cuVS retained attempts failed or returned recall 0.9. These cells were not replaced with successful samples.

E Android soak cell inventory

All cells use $100k \times 384$, $k = 10$, 512 samples, and report recall@10=1.0.

Cell	Batch	P50	P95	P99 (ms)
CPU exact	1	25.799	27.019	27.572
WGPU exact	1	14.274	17.678	25.769
WGPU exact	8	10.290	11.131	12.388
IVF-Flat	1	13.977	21.453	24.549
IVF-SQ8	1	27.359	37.090	40.581
auto CPU, 1%	1	.357	.544	.743
auto WGPU, 1%	8	8.174	9.890	12.441